

Panduan Dosen Lab 11 - RAG lokal dengan pgvector

Repo materi: [SeedFlora/meet11CloudService](#) · **durasi contoh:** 120 menit · **jalur wajib:** Docker Compose + Python hash embedding/ekstraktif tanpa akun dan tanpa API berbayar. Jalur semantic model, Groq, Supabase, dan Vercel adalah diskusi/eksperimen opsional yang memiliki kebutuhan perangkat, akun, serta keamanan tersendiri.

Hasil belajar dan kasus kerja

Mahasiswa menelusuri dokumen → chunk → vector 384 dimensi → pgvector/HNSW → top-K sumber → jawaban bersitasi. Kasus kerja adalah asisten pengetahuan internal untuk staf operasi: jawaban harus dapat dilacak ke file sumber, dan endpoint readiness harus menunjukkan ketika indeks belum siap. Bedakan retrieval yang menemukan teks dekat dari verifikasi bahwa isi jawaban benar. Mode `hash` adalah baseline leksikal untuk latihan, bukan embedding semantik produksi.

Preflight sebelum kelas

1. Periksa Docker Engine, Compose, Python 3, port 5433/8011, ruang disk, dan file `docs/*.md`. Dari root repo ini, salin `secrets/db_password.txt.example` ke `secrets/db_password.txt` dan `.env.example` ke `.env` **hanya bila file nyata belum ada**. File nyata diabaikan Git. Jangan menampilkan password di slide/terminal rekaman.
2. Jalankan `docker compose config --quiet`, lalu `docker compose up -d --wait` dan `docker compose ps`. Kunci: service `db` healthy di host `127.0.0.1:5433`. Bila port dipakai stack lain, hentikan stack itu atau ubah port di Compose dan `.env` secara konsisten.
3. PowerShell: `python -m venv .venv; .\venv\Scripts\python -m pip install -r requirements.txt`. Bash: `python3 -m venv .venv; .venv/bin/python -m pip install -r requirements.txt`.
4. Jalankan `.\venv\Scripts\python rag.py ingest` dan `.\venv\Scripts\python rag.py ask "Mengapa RLS penting untuk Supabase?"`. Kunci korpus bawaan: **3 file, 6 chunk, hash 384, chunk size 500, overlap 50**. Jawaban mode ekstraktif punya sitasi dan sumber.
5. Di terminal kedua jalankan `.\venv\Scripts\python -m uvicorn app:app --host 127.0.0.1 --port 8011`. Cek `/health`, `/ready`, `/docs`, dan `web /`. Jalankan `.\venv\Scripts\python -B tests\challenge.py`; kunci **10 PASS, 0 FAIL**.

Komputer verifikasi menjalankan image `pgvector/pgvector:pg16` healthy, Python dependencies sesuai `requirements.txt`, ingest 3/6, UI, API, benchmark 15 sampel, dan checker 10 PASS. Nilai latensi dapat berubah antar mesin.

Jadwal pembelajaran

Menit	Demo dosen	Tugas mahasiswa	Bukti
0-15	Gambar pipeline dan bedakan hash/semantic.	Baca tiga dokumen contoh.	Sebutkan unit data dan sumber.
15-30	Start Compose, lihat health DB.	Jalankan Docker lokal, catat port.	db healthy, 5433→5432.
30-45	Ingest dan baca output 3/6/500/50.	Jalankan ingest, buka SQL documents.	Jumlah chunk dan mode.
45-60	CLI ask dan urutan sumber.	Tanya dua topik, baca jarak.	Jawaban [51] dan file.
60-75	API/web, /health vs /ready, invalid 422.	Kirim dua chat, buka /docs, uji top_k:0.	Dua respons dan status.
75-90	Tambah dokumen aman, ingest ulang.	Bandingkan jumlah chunk/sumber.	Misalnya 4/7.
90-105	Jalankan benchmark 5 × 3 dan checker.	Catat p50/p95, 10 PASS.	JSONL lokal dan ringkasan.
105-120	Bahas keamanan cloud, laporan, Git.	Jawab empat pertanyaan, push.	Repo pribadi tanpa secret.

Kunci demo per tahap

A. Docker. `docker compose up -d --wait; docker compose ps`. Compose memasang volume data, init SQL, dan secret file. `01_rag.sql` membuat extension vector, tabel `documents`, metadata indeks, indeks HNSW cosine, dan fungsi `match_documents`. Pada dataset kecil planner bisa memilih sequential scan; jangan menyimpulkan HNSW rusak tanpa analisis dataset/EXPLAIN. Jangan gunakan `down -v` untuk cleanup kelas karena volume menyimpan indeks.

B. Ingest. `.\.venv\Scripts\python rag.py ingest` membaca `docs/*.md/.txt`, memotong dengan batas 500 token demo plus overlap 50, menghitung hash vector 384 dimensi, dan mengganti isi indeks dalam transaksi DB. Kunci bawaan **3 file/6 chunk**. Ini sesuai metadata `rag_meta` agar mode query dapat dibandingkan dengan mode saat indeks dibuat. Bila `.env` diubah ke `EMBED_MODE=semantic`, mahasiswa wajib ingest ulang; jangan mencampur vector dari model berbeda hanya karena dimensinya sama.

C. Liveness/readiness. `GET /health` hanya membuktikan proses HTTP hidup. `GET /ready` menanyakan DB, jumlah chunk, dan nama mode embedding; hasil sehat `status=ready` dan `chunks>0`. Jika DB mati, kosong, atau mode berubah tanpa ingest, readiness 503. Kode tidak mengembalikan password/connection string pada JSON sukses.

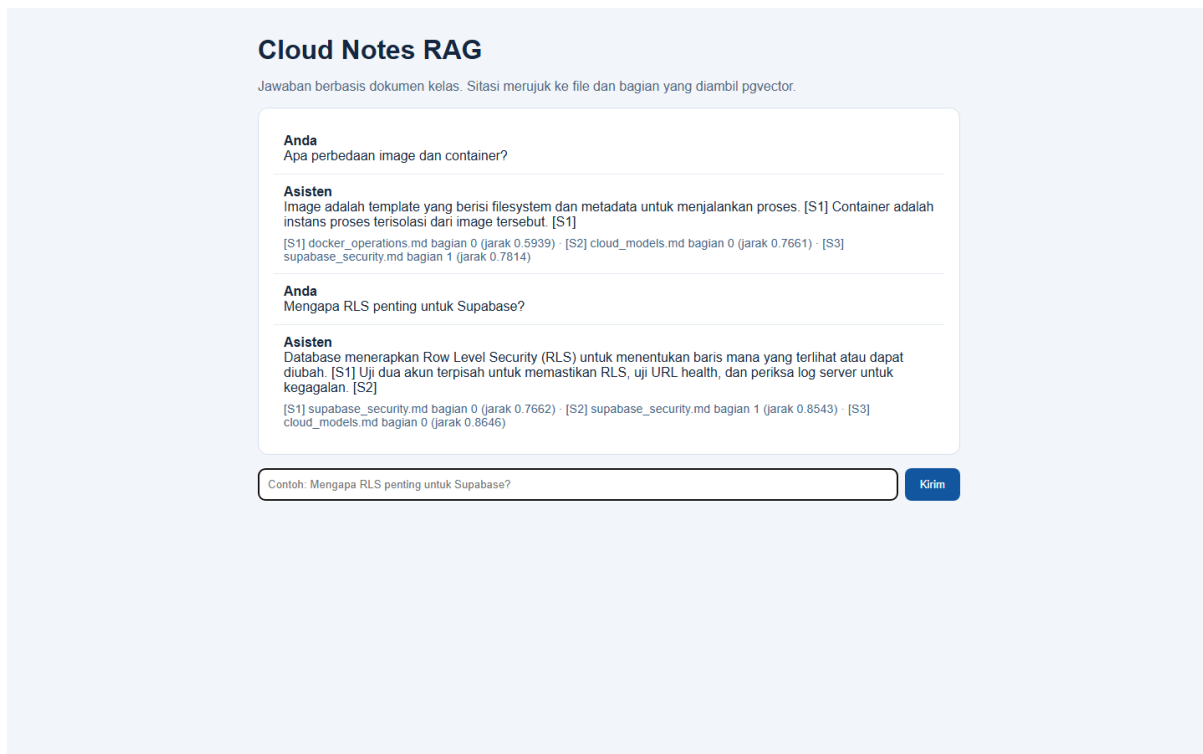
D. API dan UI. `POST /api/ask` dengan `{"question":"Apa perbedaan image dan container?","top_k":3}` memberi jawaban ekstraktif, `mode=extractive`, tiga sumber, `ref=S1...S3`, nama file, bagian, jarak. Web `index.html` menaruh respons dengan `textContent` sehingga teks dokumen tidak dieksekusi sebagai HTML. Ajukan pertanyaan kedua tentang RLS dan minta mahasiswa cocokkan klaim dengan `docs/supabase_security.md`. `top_k:0` memberi HTTP **422** dari validasi Pydantic. Sitasi adalah petunjuk sumber, bukan jaminan kebenaran.

E. Benchmark. `.\.venv\Scripts\python benchmark.py --repeat 5` menghasilkan **15 baris** di `benchmarks.jsonl` serta ringkasan p50 median/p95 nearest-rank. Uji contoh pada komputer verifikasi menghasilkan p50 61,51 ms, p95 189,26 ms; angka itu tidak menjadi syarat nilai. Diskusikan cold/warm, ukuran dokumen, dan jumlah pengulangan. File JSONL diabaikan Git.

F. Dokumen tambahan. Buat satu Markdown aman di `docs/`, ingest ulang, contoh hasil 4 file/7 chunk. Tanya topik baru; sumber bisa muncul atau tidak sesuai hash/top-K. Jika file latihan sementara dihapus, ingest ulang ke baseline

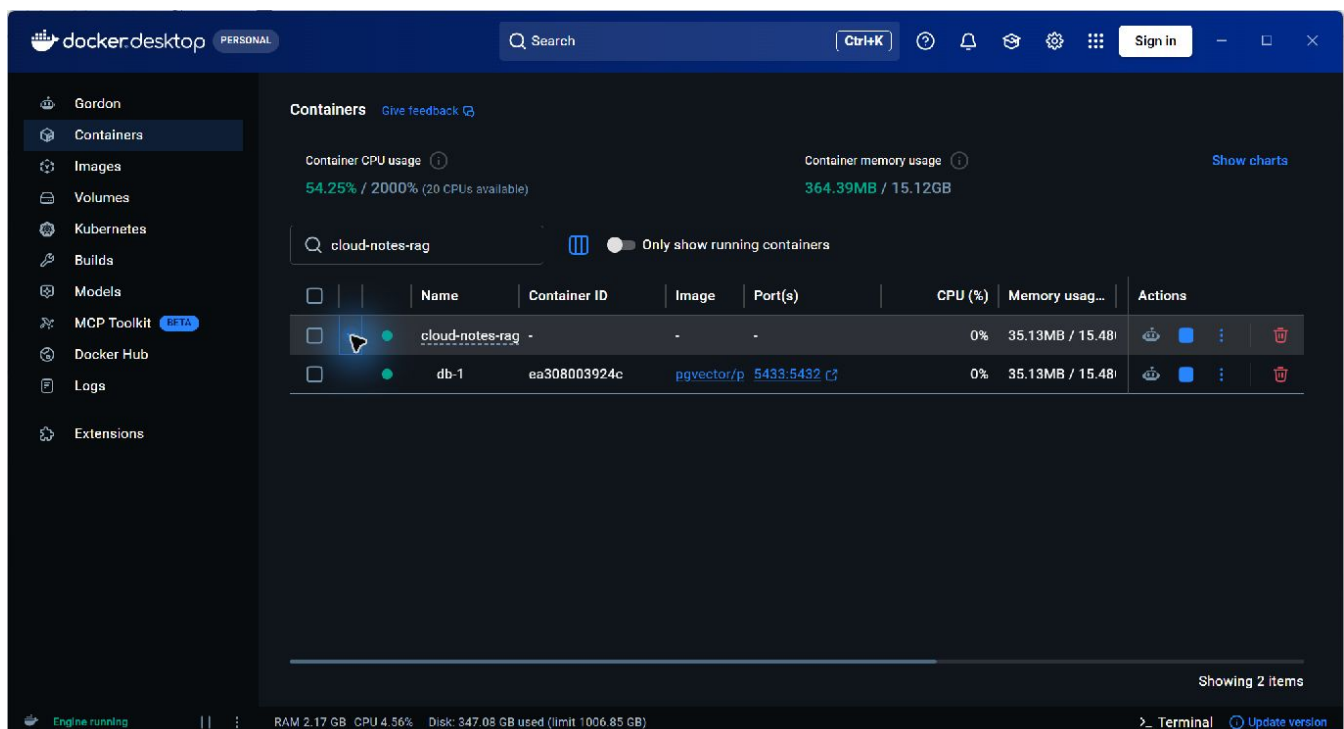
3/6. Jangan memakai dokumen pribadi mahasiswa atau kredensial dalam repo publik.

G. Checker. `.\.venv\Scripts\python -B tests\challenge.py` memeriksa vektor hash/chunk, fungsi p95, health/ready, jawaban dan tiga sumber, referensi file, mode ekstraktif, validasi 422, sitasi. **10 PASS, 0 FAIL** menunjukkan jalur lokal siap; kualitas jawaban tetap harus dinilai manusia.



UI RAG dua pertanyaan dari server lokal

Command/tindakan: jalankan Uvicorn dan kirim dua pertanyaan di web. **Fungsi:** uji chat dan sitasi end-to-end. **Cara kerja:** API membuat vector query, memanggil pgvector top-K, lalu menyusun kalimat dari chunk. **Baca hasil:** [S1] dalam kalimat dan nama/jarak sumber di bawahnya.



Docker Desktop menampilkan database pgvector sehat

Command: `docker compose up -d --wait`, lalu buka Docker Desktop Containers. **Fungsi:** membuktikan dependensi DB hidup. **Cara kerja:** Compose menjalankan image pgvector dan memetakan host 5433 ke container PostgreSQL 5432. **Baca hasil:** `cloud-notes-rag/db-1` hijau/healthy; Uvicorn 8011 berjalan terpisah pada host, jadi tidak tampil sebagai container.

Lab 11 | keluaran perintah yang diuji

```
$ docker compose ps
```

NAME	IMAGE	COMMAND	SERVICE	CREATED
cloud-notes-rag-db-1	pgvector/pgvector:pg16	"docker-entrypoint.s..."	db	15 minutes ago Up
15 minutes (healthy)	127.0.0.1:5433->5432/tcp			

Cuplikan output aktual; ditata ulang agar terbaca pada modul.

Keluaran Docker Compose Lab 11

Command: `docker compose ps`. **Fungsi:** memverifikasi keadaan yang sama melalui terminal. **Cara kerja:** Compose membaca container, image, status, port. **Baca hasil:** pgvector healthy, port 127.0.0.1:5433→5432; output aktual ditata ulang.

Lab 11 | keluaran perintah yang diuji

```
$ .\.venv\Scripts\python rag.py ingest
```

```
{
  "files": 3,
  "chunks": 6,
  "embedding_model": "hashing-384-classroom-baseline",
  "chunk_size": 500,
  "overlap": 50
}
```

Cuplikan output aktual; ditata ulang agar terbaca pada modul.

Keluaran ingest korpus bawaan

Command: `.\.venv\Scripts\python rag.py ingest`. **Fungsi:** membuat indeks. **Cara kerja:** file Markdown menjadi 6 chunk/vektor dan metadata indeks. **Baca hasil:** 3 file, 6 chunk, hashing 384, 500/50; keluaran aktual ditata ulang.

Cloud Notes RAG 0.11 OAS 3.1

/openapi.json

default

GET	/	Index	⌵
GET	/health	Health	⌵
GET	/ready	Ready	⌵
POST	/api/ask	Ask	📄 ⌵

Schemas

AskInput	Expand all	object
ChatTurn	Expand all	object
HTTPValidationError	Expand all	object
ValidationError	Expand all	object

Dokumentasi OpenAPI lokal

Command/tindakan: buka `/docs`. **Fungsi:** memperlihatkan kontrak API yang sama dengan UI/terminal. **Cara kerja:** FastAPI menghasilkan OpenAPI dari route dan model Pydantic. **Baca hasil:** `/health`, `/ready`, `POST /api/ask`, batas `top_k`.

Lab 11 | keluaran perintah yang diuji

```
$ .\.venv\Scripts\python benchmark.py --repeat 5
```

```
61.51 ms | sumber: 3 | Apa perbedaan image dan container?
53.18 ms | sumber: 3 | Apa perbedaan image dan container?
137.77 ms | sumber: 3 | Apa perbedaan image dan container?
122.70 ms | sumber: 3 | Apa perbedaan image dan container?
175.63 ms | sumber: 3 | Apa perbedaan image dan container?
189.26 ms | sumber: 3 | Mengapa Row Level Security penting untuk Supabase?
95.78 ms | sumber: 3 | Mengapa Row Level Security penting untuk Supabase?
122.32 ms | sumber: 3 | Mengapa Row Level Security penting untuk Supabase?
118.37 ms | sumber: 3 | Mengapa Row Level Security penting untuk Supabase?
50.73 ms | sumber: 3 | Mengapa Row Level Security penting untuk Supabase?
44.21 ms | sumber: 3 | Bagaimana healthcheck dan volume membantu Docker Compose?
43.51 ms | sumber: 3 | Bagaimana healthcheck dan volume membantu Docker Compose?
51.43 ms | sumber: 3 | Bagaimana healthcheck dan volume membantu Docker Compose?
48.64 ms | sumber: 3 | Bagaimana healthcheck dan volume membantu Docker Compose?
44.64 ms | sumber: 3 | Bagaimana healthcheck dan volume membantu Docker Compose?
```

Sampel: 15; p50=61.51 ms; p95=189.26 ms

Hasil: D:\cloud services\Cloud_Services_All_Labs_Siap_Push\labs\11\benchmarks.jsonl

Cuplikan output aktual; ditata ulang agar terbaca pada modul.

Benchmark berulang dan p50/p95

Command: `.\.venv\Scripts\python benchmark.py --repeat 5`. **Fungsi:** memperlihatkan distribusi latensi. **Cara kerja:** 15 panggilan RAG dicatat; median dan nearest-rank p95 dihitung. **Baca hasil:** 15 sampel, p50/p95 pada mesin uji; keluaran aktual ditata ulang.

Lab 11 | keluaran perintah yang diuji

```
$ .\.venv\Scripts\python -B tests\challenge.py
```

```
PASS embedding hash 384 dimensi  
PASS chunk mempunyai awal dan akhir valid  
PASS p95 nearest rank konsisten  
PASS liveness API HTTP 200  
PASS readiness memeriksa indeks  
PASS pertanyaan menghasilkan jawaban dan sumber  
PASS referensi sumber dapat dilacak  
PASS mode ekstraktif tidak memakai API berbayar  
PASS top_k nol ditolak HTTP 422  
PASS jawaban ekstraktif punya sitasi  
challenge: 10 PASS, 0 FAIL
```

Cuplikan output aktual; ditata ulang agar terbaca pada modul.

Checker RAG lokal

Command: `.\.venv\Scripts\python -B tests\challenge.py`. **Fungsi:** verifikasi akhir jalur lokal. **Cara kerja:** checker membaca DB/API dan fungsi utilitas. **Baca hasil:** 10 PASS, 0 FAIL; keluaran aktual ditata ulang.

Kunci tanya jawab dan keamanan

1. Hashing kata menghasilkan kemiripan leksikal, tidak mempelajari sinonim/konteks seperti embedding semantik; dipakai agar kelas berjalan tanpa unduhan model.
2. Chunk lebih kecil dapat memisahkan fakta yang saling terkait; chunk lebih besar dapat memasukkan noise dan melampaui batas model. Overlap menjaga konteks di batas dengan biaya duplikasi dan retrieval.
3. Jarak cosine adalah kedekatan representasi, bukan verifikasi fakta. Periksa isi dokumen, sitasi, pertanyaan acuan, dan kasus tanpa jawaban.
4. SQL lokal belum membuat RLS/grants per pemilik dokumen. Untuk Supabase publik, tetapkan hak akses sebelum mengunggah dokumen; simpan key DB/Groq hanya di server. Jangan langsung mengumumkan endpoint pada internet terbuka.

Penilaian dan troubleshooting

Nilai Docker/ingest (20%), retrieval/sumber (25%), API/UI + validasi (20%), benchmark/evaluasi (20%), keamanan/Git (15%). Jika Compose sehat tetapi ingest gagal, cek file `.env`, secret file, port 5433, dan log DB; jangan mencetak password. Jika `/health` 200 tetapi `/ready` 503, indeks mungkin belum diisi atau DB/mode tidak cocok. Jika jawaban tidak relevan, periksa isi sumber dan batas hash baseline; menambah `top_k` tanpa analisis dapat menambah noise. Jika p95 berbeda, jelaskan mesin, beban, dan jumlah sampel. Setelah kelas, hentikan Uvicorn dengan `Ctrl+C` dan `docker compose down` tanpa `-v`.

Bukti visual eksperimen dokumen

Lab 11 — tambah dokumen dan ingest ulang (hasil aktual)

```
$ daftar docs/*.md
```

```
cloud_models.md
docker_operations.md
kelas_visual.md
supabase_security.md
```

```
$ python rag.py ingest
```

```
{
  "files": 4,
  "chunks": 7,
  "embedding_model": "hashing-384-classroom-baseline",
  "chunk_size": 500,
  "overlap": 50
}
```

Satu dokumen baru menaikkan jumlah chunk

Command: buat satu .md aman di docs/, lalu `.\.venv\Scripts\python rag.py ingest`. **Fungsi:** menunjukkan perubahan korpus. **Cara kerja:** ingest membaca file baru dan membangun ulang indeks dalam transaksi. **Baca hasil:** contoh praktik sebelumnya naik dari 3/6 ke 4/7; jumlah pada dokumen mahasiswa dapat berbeda.